



دانشگاه صنعتی شریف

دانشکده مهندسی کامپیوتر

آز اینترنت اشیا

گزارش پیشرفت هفته دوم

پروژه ۱: پایش علائم حیاتی

درس ضابطی ۴۰۳۱۷۱۰۵۱

آراد ایزدی دوست ۴۰۳۱۰۵۷۵۸

یاسمن فرخی ۴۰۳۱۱۰۴۰۹

۱ مقدمه

در هفته‌ی اول، شبیه‌ساز علائم حیاتی در زبان پایتون توسعه داده شد و با استفاده از یکی از پروژه‌های نمونه‌ی رسمی STM32CubeF7 یک رابط کاربری اولیه روی برد نمایش داده شد. در ادامه، علاوه بر اعمال تغییرات لازم در شبیه‌ساز پایتون، تصمیم گرفته شد برای ساده‌تر شدن توسعه‌ی رابط گرافیکی، مدیریت نمایش از پیاده‌سازی مستقیم با LTDC و توابع HAL به فریم‌ورک LVGL منتقل شود. در این ساختار، مدیریت اشیای گرافیکی، تصاویر و انیمیشن‌ها توسط LVGL انجام می‌شود و LTDC تنها وظیفه‌ی انتقال محتوای Frame Buffer به نمایشگر را بر عهده خواهد داشت. هدف این تغییر، کاهش وابستگی برنامه به مدیریت مستقیم لایه‌های LTDC و ساده‌تر کردن توسعه‌ی رابط کاربری بود. همچنین این معماری امکان توسعه‌ی بخش گرافیکی در مراحل بعدی پروژه را فراهم می‌کند.

۱.۱ هدف اولیه‌ی پروژه

هدف این بخش از پروژه، طراحی و پیاده‌سازی رابط کاربری گرافیکی دستگاه پایش علائم حیاتی بر روی برد STM32F746G- DISCOVERY است. رابط کاربری باید بتواند پارامترهایی از جمله موارد زیر را نمایش دهد:

- ضربان قلب (Heart Rate)

- درصد اشباع اکسیژن خون (SpO_2)

- دمای بدن

- سیگنال ECG

در مراحل اولیه، برای بررسی قابلیت نمایش تصاویر و رابط گرافیکی، ابتدا روش‌های مختلف پیاده‌سازی رابط کاربری بررسی شدند.

۲.۱ بررسی TouchGFX

در نخستین مرحله، استفاده از TouchGFX به‌عنوان چارچوب رابط گرافیکی بررسی شد. برای این منظور، روند نمایش یک تصویر با نام Gauge.png در TouchGFX Designer بررسی شد. روند کلی پیاده‌سازی به صورت زیر بود:

۱. ایجاد پروژه در STM32CubeMX/CubeIDE

۲. فعال‌سازی Middleware مربوط به TouchGFX

۳. افزودن تصویر در TouchGFX Designer

۴. قرار دادن Image Widget روی صفحه

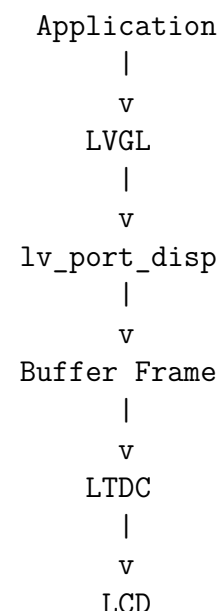
۵. تولید کد با استفاده از Generate Code

۶. ساخت پروژه با Build

با وجود امکان استفاده از این روش، به دلیل ترجیح تیم و همچنین مناسب‌تر بودن ساختار LVGL برای ادامه‌ی پروژه، تصمیم گرفته شد مسیر توسعه به سمت LVGL تغییر کند.

۳.۱ انتقال به LVGL

تفاوت اصلی LVGL با TouchGFX در نحوه‌ی توسعه‌ی رابط گرافیکی است. در حالی که TouchGFX دارای محیط طراحی گرافیکی اختصاصی است، LVGL بیشتر مبتنی بر کدنویسی است و برای استفاده روی سخت‌افزار موردنظر نیاز به پیاده‌سازی بخش‌هایی مانند Display Driver، Tick Handler و تبدیل تصاویر به آرایه‌های زبان C دارد. معماری موردنظر پس از انتقال به LVGL به صورت زیر در نظر گرفته شد:



در این ساختار، مدیریت اشیای گرافیکی، تصاویر و انیمیشن‌ها توسط LVGL انجام می‌شود و LTDC تنها محتوای موجود در Frame Buffer را به نمایشگر منتقل می‌کند.

۴.۱ راه‌اندازی اولیه‌ی پروژه در CubeMX

پروژه با استفاده از Board Selector ایجاد شد و تنظیمات اولیه‌ی برد مورد بررسی قرار گرفت. پریفرال‌های زیر به صورت فعال در پروژه وجود داشتند:

- CORTEX-M7
- GPIO
- NVIC
- DMA2D
- LTDC
- CRC

همچنین FMC با یک هشدار به شکل مثلث زرد نمایش داده می‌شد. تنظیمات SDRAM از جمله Bank 1 و CAS Latency = 3 بررسی شد و مشخص شد این مقادیر با تنظیمات استاندارد برد سازگار هستند؛ بنابراین هشدار موجود مشکل اساسی ایجاد نمی‌کرد.

در بخش Clock Configuration نیز مقدار HCLK بر روی 200 MHz تنظیم شده بود که با حداکثر فرکانس مجاز STM32F746 مطابقت داشت.

۵.۱ مشکل نصب Pack Software LVGL

در هنگام بررسی گزینه‌های Middleware در CubeMX مشخص شد که گزینه‌ی مستقیمی برای LVGL در فهرست پرفرال‌ها وجود ندارد. به همین دلیل نصب آن از طریق بخش Manage Software Packages بررسی شد. در ابتدا فهرست بسته‌های STMICROELECTRONICS ناقص بود و تنها سه بسته نمایش داده می‌شد. مشخص شد که فهرست بسته‌ها به‌روزرسانی نشده است. هنگام اجرای Refresh خطای زیر مشاهده شد:

download updaters file: unknown host server name

با توجه به شرایط دسترسی شبکه از ایران، احتمال داده شد که مشکل مربوط به عدم دسترسی به سرورهای STMICROELECTRONICS باشد.

۶.۱ مشکل دریافت Package Firmware

هنگام اجرای Generate Code خطای دیگری مربوط به در دسترس نبودن بسته‌ی Firmware مشاهده شد. نسخه‌ی موردنیاز:

STM32Cube FW_F7 V1.17.4

بود، اما CubeMX قادر به برقراری اتصال به سرور ST برای دریافت این بسته نبود. تلاش برای استفاده از گزینه‌ی Set Connection نیز به دلیل نیاز به Proxy و محدودیت دسترسی موفقیت‌آمیز نبود.

۷.۱ تلاش برای دریافت دستی STM32CubeF7

در ادامه، دریافت دستی فایل‌های موردنیاز از GitHub بررسی شد. بسته‌ی LVGL با موفقیت دریافت شد. همچنین تلاش شد STM32CubeF7 از GitHub دریافت و از طریق گزینه‌ی From Local در CubeMX اضافه شود. با این حال، گزینه‌ی From Local تنها فایل‌هایی با فرمت zip یا pack را قبول می‌کرد و پوشه‌ی معمولی را نمی‌پذیرفت. پس از تغییر نام و فشرده‌سازی مجدد پوشه‌ی دریافت‌شده، خطای زیر مشاهده شد:

file is corrupted or not a recognized package

بررسی مشخص کرد که ساختار فایل فشرده‌شده‌ی دریافت‌شده از GitHub با ساختار استاندارد CMSIS-Pack/ST مطابقت ندارد؛ بنابراین امکان استفاده‌ی مستقیم از آن به‌عنوان یک Package Software در CubeMX وجود نداشت.

۸.۱ بررسی پروژه‌های نمونه

به‌عنوان راه‌حل جایگزین، گزینه‌ی Start My Project from Example بررسی شد. با این حال، نمونه‌ی مناسبی که بتواند تمام نیازهای این پروژه و برد موردنظر را پوشش دهد پیدا نشد و این مسیر کنار گذاشته شد.

۹.۱ مشکل Import پروژه در CubeIDE

در ادامه، تلاش شد پروژه‌ای که به‌صورت ناقص توسط CubeMX تولید شده بود، با انتخاب گزینه‌ی Continue وارد STM32CubeIDE شود. این روش نیز با خطای:

Could not import project

مواجه شد.

به‌عنوان راه‌حل پیشنهادی، قرار شد پوشه‌های زیر به‌صورت دستی از فایل‌های دریافت‌شده‌ی GitHub به پروژه اضافه شوند:

Drivers/CMSIS •

Drivers/STM32F7xx_HAL_Driver •

۱۰.۱ تغییر روش توسعه و تقسیم کار

با توجه به مشکلات موجود در دریافت Firmware و همچنین در اختیار داشتن فریم ور کامل و برد فیزیکی توسط یکی از اعضای گروه، تصمیم گرفته شد که فرایند Build و Flash روی برد توسط هم تیمی انجام شود. در این ساختار، کدنویسی بخش گرافیکی در VSCode انجام شده و تغییرات از طریق Git به پروژه اصلی منتقل می شوند. این روش باعث شد توسعه ی کد از مشکلات مربوط به محیط CubeMX و دریافت خودکار Firmware مستقل شود.

۱۱.۱ وضعیت اولیه ی رابط گرافیکی

در نسخه ی اولیه ی برنامه، نمایشگر دارای وضوح:

$$480 \times 272$$

بود و Frame Buffer در آدرس:

$$0xC0000000$$

قرار داشت.

برای نمایش تصویر پس زمینه از یک لایه ی LTDC با فرمت RGB565 استفاده می شد. تصویر Splash به عنوان یک لایه ی جداگانه نمایش داده می شد و مشخصات آن به صورت زیر بود:

80, 76	Position
320 × 120	Size
RGB565	Format

پس از نمایش Splash به مدت 1.5 ثانیه، لایه ی مربوط به انیمیشن قلب فعال می شد. انیمیشن قلب شامل هشت فریم بود:

HeartFrame1 تا HeartFrame8

ابعاد هر فریم:

$$140 \times 140$$

و موقعیت آن:

$$X = 20, \quad Y = 66$$

بود.

برای قلب از فرمت ARGB4444 استفاده شده بود تا شفافیت تصویر حفظ شود. زمان نمایش فریم ها نیز به شکل زیر بود:

ms ۹۰	Frame 1-7
ms ۱۸۰	Frame 8

در نسخه ی اولیه، تغییر فریم مستقیماً با تابع:

HAL_LTDC_SetAddress()

انجام می شد.

۱۲.۱ انتقال مدیریت تصاویر به LVGL

پس از انتقال پروژه به LVGL تصمیم گرفته شد مدیریت تصاویر و انیمیشن از LTDC به LVGL منتقل شود. در نسخه‌ی جدید، تصویر Splash به جای استفاده از یک LTDC Layer به عنوان یک LVGL Image Object ایجاد شد. به همین ترتیب، قلب نیز با یک LVGL Image Object نمایش داده شد و تغییر فریم با تغییر منبع تصویر انجام گرفت. در نتیجه، مدیریت مستقیم آدرس فریم‌های مختلف در LTDC دیگر مورد نیاز نبود.

۱۳.۱ بررسی فایل‌های Converter Image

برای تبدیل تصاویر از png به فرمت مورد استفاده LVGL از وبسایت زیر استفاده شد که قابلیت آپلود عکس و انتخاب فرمت رنگ را دارد:

<https://lvgl.io/tools/imageconverter>

در هنگام تبدیل تصاویر، مشخص شد فایل‌های تولید شده توسط LVGL Image Converter از ساختار مربوط به LVGL 9 استفاده می‌کنند. برای مثال، فایل تولید شده شامل ساختاری مانند زیر بود:

```
{ = Heart1 lv_image_dsc_t const
, LV_COLOR_FORMAT_ARGB8888 = .header.cf
, LV_IMAGE_HEADER_MAGIC = .header.magic
, 882 = .header.w
, 785 = .header.h
, 4 * 692370 = .data_size
, Heart1_map = .data
};
```

وجود ساختار lv_image_dsc_t و LV_COLOR_FORMAT_ARGB8888 نشان داد که باید از API‌های مربوط به LVGL 9 استفاده شود. بنابراین به جای API‌های قدیمی مانند:

```
lv_img_create()
lv_img_set_src()
lv_img_dsc_t
```

از API‌های جدید مانند:

```
lv_image_create()
lv_image_set_src()
lv_image_dsc_t
LV_IMAGE_DECLARE()
```

استفاده شد.

۱۴.۱ مشکل اندازه‌ی تصاویر

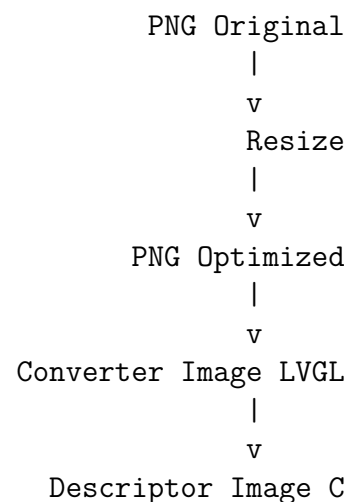
یکی از مشکلات مهم، ابعاد بسیار بزرگ تصاویر اولیه بود. برای نمونه، تصویر قلب دارای ابعاد:

$$882 \times 785$$

بود، در حالی که اندازه‌ی موردنیاز برای نمایش قلب:

$$140 \times 140$$

در نظر گرفته شده بود. مشخص شد که تغییر دستی مقادیر width و height در فایل C راه‌حل صحیحی نیست، زیرا آرایه‌ی داده‌ی تصویر همچنان مربوط به تصویر اصلی خواهد بود. بنابراین تصاویر ابتدا در قالب PNG تغییر اندازه داده شدند و سپس مجدداً توسط LVGL Image Converter به فایل‌های C تبدیل شدند. روند کلی به شکل زیر بود:



۱۵.۱ انتخاب فرمت تصاویر

در نسخه‌ی اولیه، فرمت قلب ARGB4444 بود؛ اما فایل‌های جدید Converter به صورت ARGB8888 تولید شدند. اگرچه خروجی نهایی نمایشگر RGB565 است، استفاده از ARGB8888 برای تصاویر دارای شفافیت مشکلی ایجاد نمی‌کند؛ زیرا LVGL تصویر را دریافت کرده و هنگام Render کردن، خروجی را به فرمت موردنظر نمایشگر تبدیل می‌کند.

۱۶.۱ ایجاد فایل images.h

برای دسترسی ساده‌تر به تصاویر، فایل: Core/Inc/images.h ایجاد شد. این فایل شامل Declaration تصاویر است که با استفاده از تابع LV_IMAGE_DECLARE() انجام می‌شود. به این ترتیب، تصاویر تولیدشده توسط Converter از داخل main.c قابل دسترسی شدند.

۱۷.۱ پیاده‌سازی درایور نمایش LVGL

برای اتصال LVGL به نمایشگر، دو فایل زیر ایجاد شدند:

lv_port_disp.h lv_port_disp.c

در فایل lv_port_disp.c مشخصات نمایشگر به صورت زیر تعریف شد:

```
480 LCD_WIDTH #define
272 LCD_HEIGHT #define
0xC0000000 LCD_FB_ADDR #define
```

و Frame Buffer به شکل زیر تعریف شد:

```
= *framebuffer uint16_t static
*)LCD_FB_ADDR; (uint16_t
```

بنابراین Frame Buffer همچنان در SDRAM و در آدرس 0xC0000000 قرار دارد.

۱۸.۱ استفاده از Rendering Partial

برای کاهش حافظه‌ی موردنیاز LVGL به‌جای اختصاص دو بافر به کل صفحه، دو بافر با ارتفاع ۴۰ پیکسل تعریف شدند:

```
40]; * buf1[LCD_WIDTH uint16_t static
40]; * buf2[LCD_WIDTH uint16_t static
```

حالت نمایش نیز به صورت:

```
LV_DISPLAY_RENDER_MODE_PARTIAL
```

تنظیم شد.

فرمت خروجی رنگ نیز:

```
LV_COLOR_FORMAT_RGB565
```

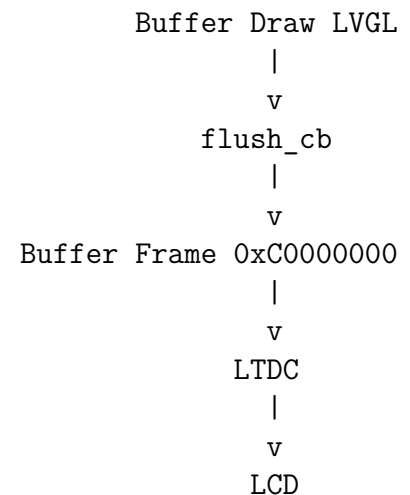
انتخاب شد.

۱۹.۱ پیاده‌سازی Callback Flush

برای انتقال داده‌ی Render شده توسط LVGL به Frame Buffer تابع flush_cb پیاده‌سازی شد.

این تابع مختصات ناحیه‌ی تغییرکرده را دریافت کرده و پیکسل‌ها را از LVGL Draw Buffer به Frame Buffer اصلی منتقل می‌کند.

مسیر داده در این قسمت به شکل زیر است:



پس از پایان انتقال نیز با دستور زیر به LVGL اعلام می‌شود که عملیات Flush به پایان رسیده است:

```
lv_display_flush_ready(disp);
```

۲۰.۱ مشکل LCD روی برد واقعی

پس از انتقال پروژه به برد واقعی، گزارش شد که هنگام نگه‌داشتن Reset صفحه سفید است و پس از رها کردن آن صفحه سیاه می‌شود. حتی تست ساده‌ی نمایش رنگ قرمز نیز مشاهده نمی‌شد. در این مرحله مشخصات زیر بررسی شدند:

- آدرس Buffer Frame برابر 0xC0000000

- آدرس Layer مربوط به LTDC برابر 0xC0000000

- تنظیمات MPU

- تنظیمات SDRAM

- تنظیمات LTDC

- Backlight

- تنظیمات LVGL

از آنجا که پروژه Build می‌شد ولی تصویر نمایش داده نمی‌شد، تصمیم گرفته شد برای جلوگیری از ترکیب چند عامل، LVGL و FreeRTOS موقتاً از تست حذف شوند.

۲۱.۱ تست Bare-Metal برای جداسازی مشکل

یک تست مرحله‌ای Bare-Metal طراحی شد تا بخش‌های مختلف زنجیره‌ی نمایش به صورت مستقل بررسی شوند. ترتیب تست به صورت زیر بود:

۱. مقداردهی اولیه‌ی FMC GPIO و LTDC

۲. تست خواندن و نوشتن SDRAM در آدرس 0xC0000000

۳. در صورت موفقیت، پر کردن کامل Buffer Frame با رنگ قرمز

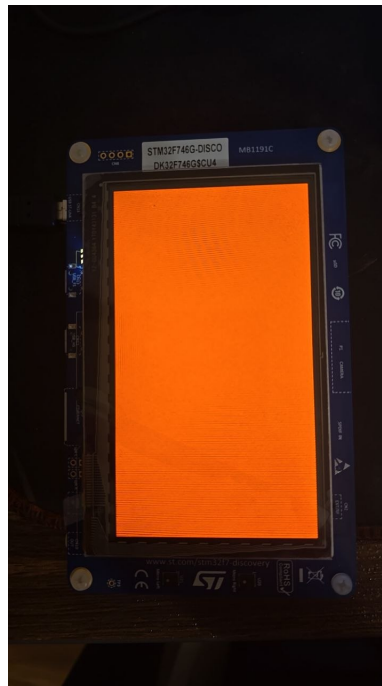
۴. اعمال مجدد آدرس Layer با HAL_LTDC_SetAddress

۵. اجرای HAL_LTDC_Reload

۶. روشن کردن پایه‌های DISP و Backlight

۷. ورود مستقیم به حلقه‌ی while(1)

۸. حذف موقت lv_init() و سایر بخش‌های LVGL



شکل ۱: تست LVLG با قرمز کردن Frame Buffer

در صورت باقی ماندن مشکل، بررسی Pixel Clock و پایه‌های RGB/HSYNC/VSYNC/DE در مقایسه با پروژه‌ی رسمی BSP به‌عنوان مرحله‌ی بعدی در نظر گرفته شد.

۲۲.۱ بررسی پروژه‌ی سالم هفته‌ی قبل

برای تشخیص اینکه مشکل از LCD یا از مسیر حافظه است، پروژه‌ی سالم هفته‌ی قبل نیز بررسی شد. این پروژه تصویر را مستقیماً از Flash به LTDC می‌داد و از LVGL و SDRAM استفاده نمی‌کرد. بنابراین سالم بودن این پروژه ثابت می‌کرد که:

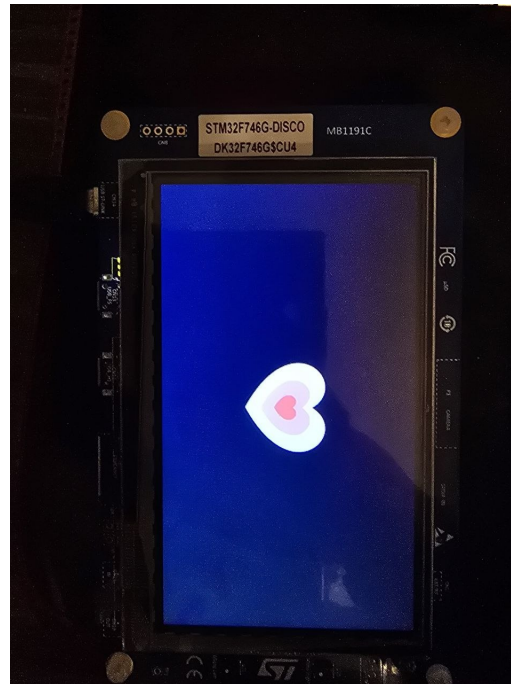
- LCD سالم است.
- Backlight سالم است.
- تنظیمات LTDC صحیح است.
- Clock Pixel صحیح است.

- پایه‌های RGB و Sync صحیح هستند.

اما این موضوع به‌تنهایی سلامت مسیر SDRAM و FMC را اثبات نمی‌کرد.

۲۳.۱ تست مستقیم Flash و LTDC

برای جدا کردن مسیر LCD از SDRAM، یک تست مستقیم طراحی شد که تصویر قلب را مستقیماً از Flash به LTDC می‌داد. در این حالت صفحه‌ی سرمه‌ای و تصویر قلب با موفقیت روی برد نمایش داده شدند.



شکل ۲: نمایش تصویر قلب روی برد

این نتیجه نشان داد که:

- LCD و Backlight سالم هستند.
- تنظیمات LTDC صحیح هستند.
- Timing نمایشگر صحیح است.
- Clock Pixel صحیح است.
- های GPIO مرتبط با LCD صحیح هستند.

بنابراین مشکل در مرحله‌ای بعد از LTDC و در مسیر FMC/SDRAM قرار داشت.

۲۴.۱ شناسایی علت اصلی با ST-LINK

برای بررسی دقیق‌تر، پردازنده در هنگام بروز خطا با استفاده از ST-LINK متوقف شد و رجیسترهای Fault بررسی شدند. مقادیر مشاهده‌شده به صورت زیر بودند:

0x080018FC	PC
0x00000082	CFSR
0xA0000140	MMFAR
Violation Access Data MemManage	نوع خطا

آدرس:

0xA0000140

در محدوده‌ی رجیسترهای کنترلی FMC قرار دارد. بررسی تنظیمات MPU نشان داد که Region 0 بخش بزرگی از فضای آدرس را با دسترسی No Access تنظیم کرده بود و در نتیجه ناحیه‌ی رجیسترهای FMC نیز توسط MPU مسدود شده بود. بنابراین اولین دسترسی تابع:

HAL_SDRAM_Init()

به رجیسترهای FMC باعث ایجاد MemManage Fault و توقف برنامه می‌شد. این بررسی علت اصلی مشکل صفحه‌ی سیاه را مشخص کرد.

۲۵.۱ اصلاح تنظیمات MPU

برای رفع مشکل، یک MPU Region جداگانه برای رجیسترهای FMC در آدرس:

0xA0000000

با اندازه‌ی ۶۴KB تعریف شد. همچنین یک MPU Region برای SDRAM در آدرس:

0xC0000000

با اندازه‌ی ۸MB ایجاد شد. این ناحیه با دسترسی کامل و بدون Cache تعریف شد تا هم CPU و هم LTDC بتوانند بدون مشکل به Frame Buffer دسترسی داشته باشند.

۲۶.۱ تست SDRAM

پس از اصلاح، MPU تست خواندن و نوشتن SDRAM با چند الگوی مختلف و چندین Offset انجام شد. همچنین در صورت شکست تست، یک صفحه‌ی بنفش به‌عنوان نشانه‌ی خطای حافظه در نظر گرفته شد. در صورت موفقیت تست، SDRAM کل Buffer Frame با رنگ قرمز پر می‌شود. این تست برای اطمینان از صحت زنجیره‌ی زیر انجام شد:

LCD → LTDC → Frame Buffer → SDRAM → FMC → CPU

۲۷.۱ بررسی FMC و SDRAM

ساختار راه‌اندازی SDRAM در فایل `fmc.c` نیز بررسی شد. ترتیب راه‌اندازی به صورت زیر تایید شد:

۱. فعال‌سازی Clock مربوط به SDRAM

۲. ایجاد تاخیر اولیه

۳. اجرای `Precharge All`

۴. اجرای هشت مرحله‌ی `Auto Refresh`

۵. بارگذاری `Mode Register`

۶. تنظیم `CAS Latency = 2`

۷. تنظیم `Refresh Rate = 0x0603`

این تنظیمات با ساختار موردنیاز SDRAM برد بررسی شدند.

۲۸.۱ اصلاح سرعت‌های LTDC GPIO

برای سیگنال‌های مربوط به LTDC در فایل `ltdc.c` سرعت‌های GPIO از مقدار `Low` به `Very High` تغییر داده شد. این تغییر برای سیگنال‌های زیر اعمال شد:

• RGB

• Clock Pixel

• HSYNC

• VSYNC

• DE

تنظیمات `Timing` و آدرس `Buffer Frame` تغییر داده نشدند و همچنان:

480×272

و:

`0xC0000000`

بودند.

۲۹.۱ تنظیم Tick مربوط به LVGL

برای عملکرد های Timer داخلی LVGL لازم است زمان سپری شده به این فریم ورک اعلام شود. از آنجا که TIM6 برای Tick سیستم استفاده می شود، در تابع:

```
HAL_TIM_PeriodElapsedCallback()
```

دستور زیر اضافه شد:

```
lv_tick_inc(1U);
```

بنابراین با هر Tick یک میلی ثانیه، زمان داخلی LVGL نیز یک میلی ثانیه افزایش می یابد. این موضوع برای عملکرد صحیح: های Timer

```
ms ۱۵۰۰    ms ۹۰    ms ۱۸۰
```

ضروری است.

همچنین مشخص شد که lv_tick_inc() نباید پیش از اجرای lv_init() فراخوانی شود.

۳۰.۱ هماهنگ سازی LVGL با FreeRTOS

از آنجا که پروژه از FreeRTOS استفاده می کند، اجرای مستقیم:

```
while(1)
{
    lv_timer_handler();
    HAL_Delay(5);
}
```

در main مناسب نبود؛ زیرا پس از:

```
osKernelStart();
```

کنترل اجرای برنامه به Scheduler منتقل می شود. بنابراین lv_timer_handler() در یک FreeRTOS Task اجرا شد. ساختار کلی این Task به شکل زیر است:

```
*argument) StartLVGLTask(void void
{
    (;;) for
    {
        lv_timer_handler();
        osDelay(5);
    }
}
```

در نسخه ی نهایی، تاخیر این Task در بازه ی ۱ تا ۲۰ میلی ثانیه نگه داشته شد تا های LVGL Timer به صورت منظم اجرا شوند.

۳۱.۱ حفظ ساختار CubeMX در main.c

از آنجا که پروژه توسط STM32CubeMX تولید شده بود، ساختار اصلی فایل main.c حفظ شد. بخش‌هایی مانند:

```
HAL_Init()
SystemClock_Config()
PeriphCommonClock_Config()
MX_GPIO_Init()
MX_FMC_Init()
MX_LTDC_Init()
MX_DMA2D_Init()
MX_FREERTOS_Init()
osKernelStart()
```

حذف نشدند.

کدهای اختصاصی پروژه تا حد امکان در بخش‌های:

```
*/ ... BEGIN CODE USER /*
*/ ... END CODE USER /*
```

قرار گرفتند تا در صورت Regenerate Code توسط CubeMX حفظ شوند.

۳۲.۱ ساختار فایل‌های بخش گرافیکی

ساختار نهایی فایل‌های مرتبط با بخش گرافیکی به صورت زیر است:

```
Core/
|
Inc/ ---+
main.h ---+ |
images.h ---+ |
lv_port_disp.h ---+ |
|
Src/ ---+
main.c ---+ |
lv_port_disp.c ---+ |
|
Images/ ---+
SplashScreen.c ---+
Heart1.c ---+
Heart2.c ---+
Heart3.c ---+
Heart4.c ---+
Heart5.c ---+
Heart6.c ---+
Heart7.c ---+
Heart8.c ---+
```

همچنین فایل‌های مربوط به تصاویر جدید داشبورد نیز به این مجموعه اضافه شدند.

۳۳.۱ بهینه‌سازی تصاویر جدید

در ادامه‌ی توسعه‌ی رابط گرافیکی، سه کارت برای نمایش پارامترهای علائم حیاتی در نظر گرفته شد:

- ضربان قلب

- دمای بدن

- درصد اشباع اکسیژن خون

برای هر کارت از تصاویر گرافیکی مناسب استفاده شد. دارایی‌های گرافیکی مورد استفاده شامل:

- Bubble

- Thermometer

- Gauge

- GaugeArrow

- هشت فریم Heart1 تا Heart8

بودند.

برخی تصاویر اولیه ابعاد بسیار بزرگی داشتند و مجموع داده‌ی خام آن‌ها بیش از ۱۱MB بود. از آنجا که حافظه‌ی Flash داخلی میکروکنترلر محدود است، تصاویر با حذف نواحی شفاف اضافی Crop و سپس با فیلتر باکیفیت Resize شدند. برای تصویر دماسنج نیز لایه‌ی سفید تصویر به Alpha تبدیل شد تا تصویر روی پس‌زمینه‌ی رابط کاربری به‌صورت صحیح ترکیب شود.

ابعاد و حجم نهایی تصاویر به صورت زیر به دست آمد:

حجم خام	ابعاد نهایی	Asset
bytes ۲۵,۶۲۰	105 × 61	Bubble
bytes ۲۳,۵۴۰	55 × 107	Thermometer
bytes ۲۸,۲۲۴	84 × 84	Gauge
bytes ۲۸,۲۲۴	84 × 84	GaugeArrow

برای Gauge و GaugeArrow دقیقاً از یک Crop و Resize مشترک استفاده شد تا موقعیت عقربه نسبت به گیج در هنگام نمایش صحیح باقی بماند.

۳۴.۱ داشبورد نهایی LVGL

در نسخه‌ی جدید رابط کاربری، صفحه‌ی اصلی در رزولوشن 480 × 272 به سه کارت اصلی تقسیم شد. مشخصات اولیه‌ی کارت‌ها به صورت زیر است:

کارت	تصاویر	مقدار اولیه
ضربان قلب	Heart1-Heart8	BPM ۷۶
دما	Bubble Thermometer	C ۰.۳۷
اکسیژن خون	GaugeArrow Gauge	۹۸%

انیمیشن قلب همچنان با همان هشت فریم قبلی اجرا می‌شود. همچنین یک تابع عمومی با نام:

Vitals_UpdateUI()

برای به‌روزرسانی مقادیر SpO2، Heart Rate، دما و نام سناریو ایجاد شد. این تابع باید از Context مربوط به LVGL Task فراخوانی شود و نباید مستقیماً از ISR اجرا شود.

۳۵.۱ شبیه‌ساز علائم حیاتی

در کنار توسعه‌ی رابط کاربری، شبیه‌ساز علائم حیاتی نیز در پوشه‌ی:

simulation

قرار گرفت.

این شبیه‌ساز داده‌های فیزیولوژیک را به‌صورت پیوسته تولید می‌کند. سناریوهای فعلی عبارت‌اند از:

• Normal

• Tachycardia

• Bradycardia

• Low_SpO2

• Abnormal_Temp

داده‌ها با نرخ پیش‌فرض ۵۰ نمونه بر ثانیه تولید می‌شوند.

۳۶.۱ قرارداد ارتباط سریال

برای ارتباط شبیه‌ساز با برد، داده‌ها به صورت JSON و در قالب newline-delimited تولید می‌شوند. یک نمونه‌ی داده به شکل زیر است:

```

{"spo2":98,"hr":76{"scenario":"Normal"
,"ecg":0.124}"temp":37.0

```

ارتباط سریال با مشخصات زیر در نظر گرفته شد:

۱۱۵۲۰۰	Rate Baud
۸	Bits Data
None	Parity
۱	Bits Stop

روی برد، رابط USART1 که به Virtual COM Port متصل است نیز با همین مشخصات تنظیم شده است. در حال حاضر داشبورد با مقادیر اولیه‌ی سناریوی Normal نمایش داده می‌شود. مرحله‌ی بعدی ارتباط، دریافت داده از UART با استفاده از Interrupt یا DMA و ایجاد یک Line Buffer تا رسیدن به کاراکتر newline است. پس از دریافت یک پیام کامل، فیلدهای Parse JSON شده و تابع Vitals_UpdateUI() برای به‌روزرسانی رابط کاربری فراخوانی خواهد شد. از آنجا که تغییر مستقیم اشیای LVGL از داخل ISR مجاز نیست، به‌روزرسانی رابط کاربری باید در Context مربوط به Task LVGL انجام شود.

۳۷.۱ مشکل وابستگی شبیه‌ساز پایتون

شبیه‌ساز در صورت نبود کتابخانه‌ی PySerial نیز می‌تواند در حالت Mock اجرا شود. برای ارسال واقعی داده از طریق پورت سریال، dependency مربوط به فایل:

simulation/requirements.txt

باید نصب شود.
نام صحیح بسته‌ی موردنیاز:

pyserial

است و نباید بسته‌ای با نام متفاوت مانند serial به‌عنوان جایگزین آن نصب شود.

۳۸.۱ فرایند Build

با توجه به اینکه Build پروژه توسط هم‌تیمی انجام می‌شود، ساخت پروژه با CMake و arm-none-eabi-gcc انجام می‌شود. از پوشه‌ی ریشه‌ی پروژه، ابتدا پوشه‌ی Build پوشه‌ی Build پاک‌سازی شده و سپس پروژه مجدداً Configure و Build می‌شود:

```
build cd
* -rf rm
-eabi.cmake-none-DCMAKE_TOOLCHAIN_FILE=../arm .. cmake
-j4 make
```

پس از تولید فایل ELF، خروجی‌های HEX و BIN با استفاده از arm-none-eabi-objcopy تولید می‌شوند. برای پروگرام برد، فایل:

build/VitalSignsMonitor.bin

و آدرس شروع Flash:

0x08000000

استفاده می‌شود.

۳۹.۱ مشخصات آخرین Build

آخرین Build تمیز پروژه با موفقیت انجام شد. مشخصات خروجی به صورت زیر است:

bytes ۱,۱۳۷,۰۳۶	ELF
bytes ۹۹۳,۷۱۶	BIN
bytes ۲,۷۹۵,۱۶۵	HEX
bytes ۹۹۳,۷۰۸	text+data
حدود ۶.۵۳ KiB	Flash فضای باقی‌مانده

همچنین مقدار SHA-256 فایل BIN به صورت زیر ثبت شد:

4b3ec127626b8620fdb201e5a9135d296a4d576c1a62c56e2eed2307ea631aa0

ثبت این مقدار امکان بررسی یکسان بودن فایل پروگرام‌شده در مراحل بعدی را فراهم می‌کند.

۴۰.۱ حالت تشخیصی و ترتیب اجرای برنامه

برای ساده‌تر شدن عیب‌یابی، نسخه‌ی تشخیصی برنامه مراحل زیر را طی می‌کند:

۱. حدود ۳ ثانیه نمایش صفحه‌ی سرمه‌ای و قلبی که مستقیماً از Flash به LTDC داده می‌شود.
۲. راه‌اندازی FMC و SDRAM.
۳. اجرای تست حافظه.
۴. نمایش رنگ بنفش در صورت شکست تست SDRAM.
۵. نمایش صفحه‌ی قرمز در صورت موفقیت تست SDRAM.
۶. راه‌اندازی LVGL.
۷. ایجاد رابط کاربری.
۸. اجرای انیمیشن قلب.

در این ساختار، صفحه‌ی سرمه‌ای ابتدای بوت توسط LVGL تولید نمی‌شود و صرفاً یک تست مستقل برای LTDC → Flash → LCD است. در مقابل، رابط نهایی و انیمیشن قلب توسط LVGL رندر می‌شوند.

۲ تغییرات شبیه‌ساز علائم حیاتی

در این مرحله، هدف اصلی تکمیل و استانداردسازی شبیه‌ساز رایانه‌ای علائم حیاتی بود تا داده‌های مربوط به وضعیت بیمار به‌شکل قابل‌اعتماد تولید شده و از طریق رابط UART برای برد STM32 ارسال شوند. علاوه بر این، ساختار نرم‌افزار به‌گونه‌ای بازطراحی شد که امکان تست، نگهداری و توسعه‌ی مستقل بخش‌های مختلف آن فراهم شده و اتصال آن به بخش Embedded پروژه با ابهام کمتری انجام شود.

۱.۲ بازطراحی و ماژولار کردن شبیه‌ساز

در نسخه‌ی قبلی شبیه‌ساز، تقریباً تمام منطق مربوط به تولید داده، تولید سیگنال ECG، تعریف سناریوها و ارتباط UART در یک فایل قرار داشت. این ساختار باعث می‌شد توسعه و تست قسمت‌های مختلف برنامه دشوار باشد. به همین دلیل، ساختار شبیه‌ساز به چند بخش مستقل تقسیم شد که هرکدام مسئولیت مشخصی دارند:

- scenarios.py: تعریف سناریوها و بازه‌ی مجاز علائم حیاتی

- patient.py: مدل بیمار، تغییرات علائم حیاتی و تولید سیگنال ECG

- uart.py: ساخت فریم‌های JSON و مدیریت ارتباط سریال

- main.py: اجرای اصلی برنامه و مدیریت ورودی‌های خط فرمان

- init.py:

با این تغییر، هر قسمت از برنامه مسئولیت مشخصی پیدا کرده و امکان توسعه و آزمایش بخش‌های مختلف به‌صورت مستقل فراهم شده است.

۲.۲ تکمیل سناریوهای بیمار

پنج سناریوی موردنیاز پروژه در شبیه‌ساز تعریف و بررسی شدند:

- فرد سالم (Normal)
- تاکی‌کاردی یا ضربان قلب بالا (Tachycardia)
- برادی‌کاردی یا ضربان قلب پایین (Bradycardia)
- کاهش سطح اکسیژن خون (Low_SpO2)
- دمای غیرعادی بدن (Abnormal_Temp)

برای هر سناریو، بازه‌ی مناسب برای ضربان قلب، سطح اکسیژن خون و دمای بدن مشخص شد. همچنین در صورت وارد کردن نام نامعتبر برای سناریو، برنامه خطای مناسب را نمایش داده و فهرست سناریوهای معتبر را در اختیار کاربر قرار می‌دهد.

۳.۲ اصلاح مدل تغییرات علائم حیاتی

یکی از مشکلات نسخه‌ی قبلی شبیه‌ساز، احتمال ایجاد پرش ناگهانی در مقادیر هنگام تغییر سناریو بود. برای مثال، با تغییر وضعیت از حالت سالم به تاکی‌کاردی، ممکن بود مقدار ضربان قلب در یک نمونه از حدود 74 به بیش از 108 برسد. چنین تغییری از نظر فیزیولوژیکی طبیعی نیست و همچنین می‌تواند باعث ایجاد نمودارهای غیرواقعی در رابط کاربری شود. برای رفع این مشکل، برای هر پارامتر یک مقدار فعلی و یک مقدار هدف در نظر گرفته شد. مقدار فعلی با سرعت کنترل‌شده به سمت مقدار هدف حرکت می‌کند. در حالت پایدار، تغییرات به آرامی انجام می‌شوند و هنگام تغییر سناریو، سرعت حرکت به سمت مقدار هدف افزایش می‌یابد؛ با این حال، تغییرات همچنان به صورت پیوسته انجام می‌شوند.

همچنین نویزهای کوچک و طبیعی به داده‌ها اضافه شد و محدودیت‌های فیزیولوژیک کلی برای جلوگیری از تولید مقادیر غیرممکن اعمال شدند. در نتیجه، تغییر وضعیت بیمار طبیعی‌تر شده و داده‌های تولیدشده برای نمایش و رسم نمودار نیز مناسب‌تر هستند.

۴.۲ تکمیل تولید سیگنال ECG

سیگنال ECG بر اساس ضربان قلب فعلی بیمار تولید می‌شود. در مدل استفاده‌شده، اجزای اصلی شکل موج ECG شامل موارد زیر در نظر گرفته شده‌اند:

- موج P
- موج Q
- قله‌ی R
- موج S
- موج T
- تغییر آهسته‌ی خط مبنا
- نویز اندازه‌گیری

مدت زمان هر سیکل ECG از رابطه‌ی زیر محاسبه می‌شود:

$$T_{\text{cycle}} = \frac{60}{HR}$$

بنابراین، با افزایش ضربان قلب، فاصله‌ی زمانی بین ضربان‌های متوالی ECG کاهش می‌یابد و در نتیجه شکل موج تولیدشده با وضعیت ضربان قلب بیمار سازگار باقی می‌ماند.

۵.۲ استانداردسازی ارسال JSON روی UART

برای ارتباط بین شبیه‌ساز و برد STM32، قالب مشخصی برای انتقال داده‌ها در نظر گرفته شد. هر نمونه به‌صورت یک فریم JSON فشرده تولید می‌شود. برای مثال:

```
[language=json] "scenario": "Normal", "hr": ۷۴, "spo۲": ۹۸, "temp": ۳۶.۸, "ecg": ۰.۰۲۱
```

ویژگی‌های ارتباطی تعریف‌شده عبارت‌اند از:

- سرعت پیش‌فرض ارتباط UART برابر با 115200 بیت بر ثانیه
- قالب ارتباطی 8N1
- کدگذاری داده‌ها به‌صورت UTF-8
- پایان هر فریم با نویسه‌ی \n
- حذف فاصله‌های اضافی از JSON برای کاهش حجم پیام
- جلوگیری از تولید مقادیر نامعتبر مانند NaN
- بسته‌شدن امن پورت سریال
- انتقال خودکار به حالت Mock در صورت بازنشدن یا قطع پورت

استفاده از نویسه‌ی پایان خط باعث می‌شود سمت STM32 بتواند داده‌ها را تا دریافت \n در یک بافر جمع‌آوری کرده و سپس کل فریم JSON را پردازش کند.

۶.۲ افزودن رابط خط فرمان

در نسخه‌ی قبلی، پورت سریال و سناریوی مورد استفاده مستقیماً در کد مشخص می‌شدند. در نسخه‌ی جدید، امکان تعیین تنظیمات مختلف از طریق خط فرمان فراهم شد و برای اجرای شبیه‌ساز دیگر نیازی به تغییر کد وجود ندارد. گزینه‌های اضافه‌شده شامل موارد زیر هستند:

- انتخاب پورت سریال
- انتخاب سناریو
- انتخاب baudrate
- انتخاب نرخ نمونه‌برداری
- تعیین مدت زمان اجرا
- تعیین تعداد فریم‌ها
- تعیین seed برای تولید داده‌های تکرارپذیر
- نمایش فهرست پورت‌های سریال موجود
- تنظیم فاصله‌ی نمایش خروجی در ترمینال

برای مثال، اجرای شبیه‌ساز با سناریوی تاکی‌کاردی و نرخ نمونه‌برداری 50 هرتز به‌صورت زیر است:

```
Tachycardia -scenario /dev/cu.usbserial-XXXX -port simulation -m python۳ [language=bash] ۵۰ -sample-rate
```

روش قبلی اجرای برنامه با استفاده از فایل main.py نیز برای حفظ سازگاری با نسخه‌های قبلی نگه داشته شد.

۷.۲ افزودن حالت آزمایشی بدون سخت افزار

از آنجا که در زمان توسعه امکان تست کامل ارتباط با برد و مبدل سریال وجود نداشت، حالت Mock تکمیل و حفظ شد. در این حالت، تمام داده ها و فریم های JSON تولید می شوند، اما اطلاعات روی سخت افزار ارسال نمی شوند. این حالت امکان آزمایش منطق شبیه ساز را بدون نیاز به اتصال فیزیکی به برد فراهم می کند. علاوه بر حالت Mock، از پورت مجازی //loop: در کتابخانه PySerial نیز استفاده شد. در این روش، برنامه فریم تولید شده را روی پورت سریال مجازی ارسال کرده و همان فریم را مجدداً دریافت می کند. به این ترتیب، ساخت JSON و عملیات نوشتن و دریافت داده از رابط سریال بدون نیاز به برد فیزیکی قابل بررسی است.

۸.۲ نوشتن تست های خودکار

برای جلوگیری از ایجاد خطا در تغییرات آینده و بررسی خودکار عملکرد بخش های مختلف شبیه ساز، هشت تست خودکار نوشته شد:

۱. بررسی قرارگیری علائم حیاتی در بازه ی تعریف شده برای هر پنج سناریو

۲. بررسی پیوستگی تغییرات هنگام تغییر سناریو

۳. بررسی رسیدن تدریجی مقادیر به بازه ی سناریوی جدید

۴. بررسی وجود قله ها و بخش های مثبت و منفی در سیگنال ECG

۵. بررسی نام و نوع فیلدهای خروجی

۶. بررسی فشردگی بودن JSON و وجود n\ در انتهای فریم

۷. تست ارسال و دریافت واقعی با استفاده از PySerial Loopback

۸. بررسی ارسال تعداد مشخصی فریم و بسته شدن صحیح ارتباط

هر هشت تست با موفقیت اجرا شدند:

OK tests 8 Ran

همچنین یک اجرای آزمایشی با استفاده از پورت مجازی، سناریوی فرد سالم و نرخ نمونه برداری 50 هرتز انجام شد و سه فریم با موفقیت ارسال شدند.

۹.۲ مستندسازی قرارداد ارتباطی

برای مشخص کردن دقیق نحوه ی ارتباط میان شبیه ساز و برد، فایل UART_PROTOCOL.md ایجاد شد. هدف از این فایل جلوگیری از ابهام در هنگام پیاده سازی بخش دریافت داده در سمت STM32 است. موارد زیر در این قرارداد مشخص شده اند:

- تنظیمات کامل UART

- قالب JSON

- نوع و واحد هر فیلد

- روش تشخیص پایان فریم

- حداقل اندازه ی بافر مورد نیاز در سمت STM32

- نحوه‌ی برخورد با فریم خراب یا ناقص
- نحوه‌ی تشخیص قطع ارتباط یا قدیمی‌شدن داده‌ها

طبق این قرارداد، سمت برد باید داده‌های دریافتی را تا رسیدن نویسه‌ی `\n` در بافر جمع‌آوری کرده و پس از دریافت یک فریم کامل، محتوای JSON را پردازش کند.

۱۰.۲ تکمیل README و نحوه‌ی اجرای پروژه

برای ساده‌تر شدن راه‌اندازی و استفاده از شبیه‌ساز، فایل README نیز بازنویسی شد. موارد زیر به مستندات اضافه شدند:

- معرفی دقیق شبیه‌ساز
- امکانات پیاده‌سازی‌شده
- مراحل ایجاد محیط مجازی Python
- نصب وابستگی‌ها
- اجرای حالت Mock
- اجرای ارتباط UART واقعی
- نمایش پورت‌های سریال موجود
- اجرای تست‌های خودکار
- نمونه‌ی فریم JSON
- تقسیم مسئولیت میان اعضای گروه
- بخش‌های باقی‌مانده برای مرحله‌ی Integration

۱۱.۲ مدیریت وابستگی‌ها و فایل‌های اضافی

برای مدیریت وابستگی‌های پروژه، فایل `requirements.txt` اضافه شد و نسخه‌ی موردنیاز کتابخانه‌ی PySerial در آن ثبت شد.

همچنین فایل `gitignore` ایجاد شد تا فایل‌های غیرضروری وارد های `commit` بعدی نشوند. مواردی که در این فایل در نظر گرفته شده‌اند شامل موارد زیر هستند:

- فایل‌های `cache` مربوط به Python
- محیط‌های مجازی Python
- فایل‌های آزمایشی و خروجی‌های مربوط به تست
- فایل‌های `.DS_Store`
- اطلاعات محلی STM32CubeIDE
- خروجی‌های `build`، `Debug` و `Release`

۱۲.۲ وضعیت فعلی بخش شبیه‌ساز

در پایان این مرحله، بخش نرم‌افزاری شبیه‌ساز از نظر تولید داده و آماده‌سازی برای اتصال به برد به وضعیت قابل‌استفاده‌ای رسید. تولید داده‌های علائم حیاتی، تعریف سناریوها، تولید سیگنال ECG، قالب‌بندی JSON، مدیریت ارتباط UART، رابط خط فرمان و تست‌های نرم‌افزاری تکمیل شده‌اند. موارد باقی‌مانده برای مرحله‌ی بعد عبارت‌اند از:

- آزمایش ارتباط با استفاده از مبدل واقعی USB-to-Serial
- مشخص کردن USART مورد استفاده در برد STM32
- بررسی دریافت صحیح فریم‌ها در سمت برد
- تنظیم نهایی نرخ نمونه‌برداری و مقیاس سیگنال ECG متناسب با نمودار رابط گرافیکی
- آزمایش هر پنج سناریو روی نمایشگر
- انجام تست کامل End-to-End از شبیه‌ساز تا نمایشگر و رفع خطاهای احتمالی ارتباطی

```
> python -m simulation \
--port /dev/cu.usbmodem2103 \
--scenario Normal \
--sample-rate 10
[!] Could not open /dev/cu.usbmodem2103; running in MOCK mode.
[*] Scenario: Normal
[*] Sample rate: 10.0 Hz
[*] Press Ctrl+C to stop.
Mock: {"scenario":"Normal","hr":75,"spo2":99,"temp":36.8,"ecg":-0.001}
Mock: {"scenario":"Normal","hr":74,"spo2":99,"temp":36.8,"ecg":0.329}
Mock: {"scenario":"Normal","hr":73,"spo2":98,"temp":36.8,"ecg":0.969}
```

شکل ۳: خروجی اجرای شبیه‌ساز به صورت mock

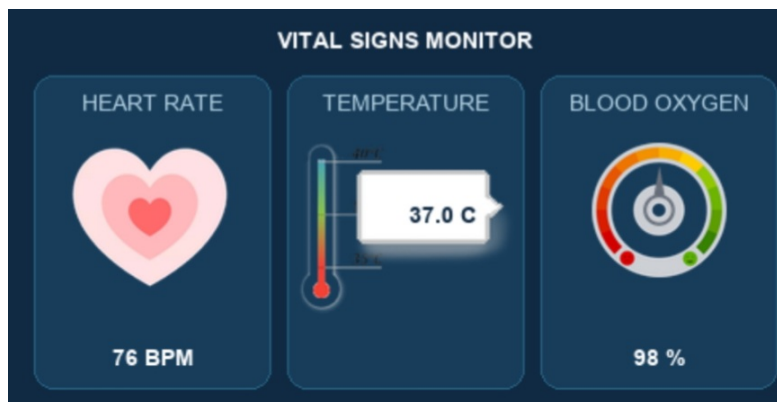
همچنین در این بخش داده‌ها به برد فرستاده می‌شوند اما هنوز در رابط گرافیکی آپدیت نمی‌شوند که در هفته آینده این بخش را کامل خواهیم کرد.

```
> python -m simulation \
--port /dev/cu.usbmodem2103 \
--scenario Normal \
--sample-rate 10
[*] Connected to /dev/cu.usbmodem2103 @ 115200 baud
[*] Scenario: Normal
[*] Sample rate: 10.0 Hz
[*] Press Ctrl+C to stop.
Sent: {"scenario":"Normal","hr":77,"spo2":98,"temp":36.7,"ecg":-0.005}
Sent: {"scenario":"Normal","hr":78,"spo2":98,"temp":36.7,"ecg":0.368}
Sent: {"scenario":"Normal","hr":79,"spo2":98,"temp":36.7,"ecg":0.002}
Sent: {"scenario":"Normal","hr":79,"spo2":98,"temp":36.7,"ecg":-0.018}
Sent: {"scenario":"Normal","hr":79,"spo2":98,"temp":36.7,"ecg":0.03}
Sent: {"scenario":"Normal","hr":79,"spo2":98,"temp":36.7,"ecg":-0.086}
Sent: {"scenario":"Normal","hr":79,"spo2":98,"temp":36.7,"ecg":0.042}
Sent: {"scenario":"Normal","hr":79,"spo2":98,"temp":36.7,"ecg":0.023}
Sent: {"scenario":"Normal","hr":79,"spo2":98,"temp":36.7,"ecg":0.056}
Sent: {"scenario":"Normal","hr":79,"spo2":98,"temp":36.7,"ecg":0.098}
Sent: {"scenario":"Normal","hr":78,"spo2":98,"temp":36.7,"ecg":-0.032}
Sent: {"scenario":"Normal","hr":77,"spo2":98,"temp":36.7,"ecg":0.073}
Sent: {"scenario":"Normal","hr":76,"spo2":98,"temp":36.8,"ecg":0.019}
```

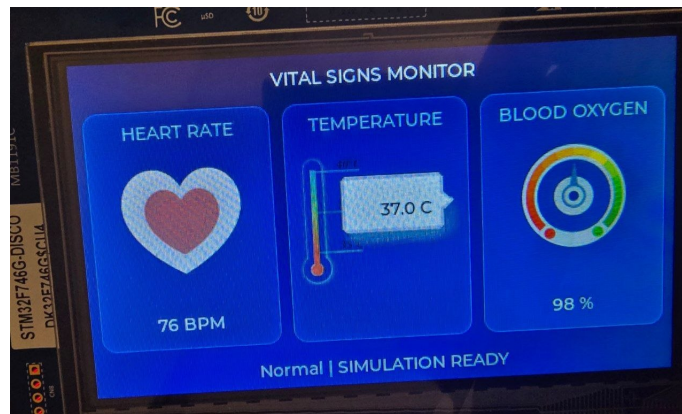
شکل ۴: خروجی اجرای شبیه‌ساز به همراه فرستادن داده‌ها به برد

۱۳.۲ وضعیت فعلی پروژه

تا اینجا رابط کاربری به صورت زیر شده است:



شکل ۵: رابط کاربری واحد پایش علائم حیاتی



شکل ۶: رابط کاربری نمایش داده شده روی برد

در پایان این مرحله، موارد زیر انجام شده یا تایید شده‌اند:

- شبیه‌ساز علائم حیاتی در پایتون ایجاد شده است.
- سناریوهای مختلف علائم حیاتی در شبیه‌ساز پیاده‌سازی شده‌اند.
- مسیر اولیه‌ی رابط گرافیکی با TouchGFX بررسی شده است.
- پروژه به استفاده از LVGL منتقل شده است.
- پورت نمایش LVGL پیاده‌سازی شده است.
- Buffer Frame در SDRAM در آدرس 0xC0000000 قرار گرفته است.
- Rendering Partial با دو بافر ۴۰ خطی پیاده‌سازی شده است.
- تابع Flush برای انتقال داده از LVGL به Buffer Frame ایجاد شده است.
- تصاویر به فرمت قابل استفاده توسط LVGL تبدیل شده‌اند.
- تصاویر بزرگ برای کاهش مصرف Flash بهینه شده‌اند.
- Screen Splash در LVGL پیاده‌سازی شده است.
- انیمیشن هشت‌فریمی قلب در LVGL پیاده‌سازی شده است.
- Tick مربوط به LVGL با TIM۶ هماهنگ شده است.
- LVGL با FreeRTOS هماهنگ شده است.
- مشکل عدم نمایش تصویر روی LCD بررسی و مرحله‌بندی شده است.
- مسیر مستقیم Flash به LTDC با موفقیت آزمایش شده است.
- علت اصلی خطای SDRAM با استفاده از ST-LINK و رجیسترهای Fault شناسایی شده است.
- تنظیمات MPU برای FMC و SDRAM اصلاح شده‌اند.
- تست‌های SDRAM و Buffer Frame اضافه شده‌اند.

- داشبورد سه گانه‌ی ضربان قلب، دما و اکسیژن خون طراحی شده است.
- تابع `Vitals_UpdateUI()` برای به‌روزرسانی مقادیر ایجاد شده است.
- قرارداد JSON برای ارتباط شبیه‌ساز و برد تعریف شده است.
- UART با تنظیمات ۱۱۵۲۰۰/۸-N-۱ برای ارتباط با شبیه‌ساز آماده شده است.
- پروژه با CMake به‌صورت موفقیت‌آمیز Build شده است.
- فایل BIN نهایی برای پروگرام برد تولید شده است.

۱۴.۲ موارد باقی‌مانده

با وجود آماده بودن بخش عمده‌ی رابط گرافیکی، چند مرحله برای ادامه‌ی توسعه باقی مانده است:

۱. پروگرام آخرین فایل BIN روی برد و بررسی نمایش صحیح هر سه کارت.
۲. تایید هم‌ترازی نهایی تصاویر دماسنج، گیج و عقربه روی نمایشگر واقعی.
۳. تکمیل دریافت داده از UART با استفاده از Interrupt یا DMA.
۴. ایجاد Line Buffer برای دریافت پیام‌های JSON.
۵. Parse کردن داده‌های دریافتی و انتقال آن‌ها به تابع `Vitals_UpdateUI()`.
۶. آزمایش سناریوهای مختلف شبیه‌ساز مانند Tachycardia، Bradycardia، Low_SpO2 و Abnormal_Temp.
۷. پس از تثبیت مسیر LCD، غیرفعال کردن LCD_BRINGUP_MODE و فعال‌سازی تدریجی سایر Peripher-های پروژه.
۸. حذف یا غیرفعال کردن تست‌های تشخیصی مانند صفحه‌ی قرمز و بنفش در نسخه‌ی نهایی، یا نگه‌داشتن آن‌ها پشت یک compile-time flag.

۱۵.۲ جمع‌بندی

مهم‌ترین دستاورد این مرحله، انتقال موفق مدیریت رابط گرافیکی از پیاده‌سازی مستقیم مبتنی بر LTDC به معماری مبتنی بر LVGL و همچنین ایجاد یک مسیر مشخص برای ارتباط شبیه‌ساز علائم حیاتی با رابط کاربری بود. در فرایند عیب‌یابی LCD نیز مشخص شد که مشاهده‌ی صفحه‌ی سیاه لزوماً به معنی خرابی نمایشگر یا LVGL نیست. با جدا کردن مسیر تصویر به چند مرحله، ابتدا مسیر LCD → LTDC → Flash به‌صورت مستقل آزمایش شد و سلامت LCD، Timing Clock، Pixel Backlight، و پایه‌های مربوط به LTDC تایید گردید. سپس با توقف پردازنده توسط ST-LINK و بررسی رجیسترهای Fault مشخص شد که خطای اصلی در مسیر FMC/SDRAM قرار دارد. مقدار MMFAR نشان داد که دسترسی به محدوده‌ی رجیسترهای FMC توسط MPU مسدود شده است. با تعریف نواحی صحیح برای رجیسترهای FMC و حافظه‌ی SDRAM، امکان استفاده‌ی صحیح از Buffer Frame خارجی فراهم شد. در نتیجه، معماری نهایی پروژه به صورت زیر شکل گرفته است:

```

Simulator Signs Vital Python
|
UART / JSON |
v
STM32F746G
|
v
Objects LVGL
|
v
Rendering RGB565 Partial
|
v
Framebuffer SDRAM
0xC0000000
|
v
LTDC
|
v
LCD

```

در حال حاضر بخش گرافیکی پروژه شامل داشبورد سه گانه‌ی ضربان قلب، دما و اکسیژن خون، انیمیشن قلب و زیرساخت دریافت داده از شبیه‌ساز آماده است و مرحله‌ی بعدی پروژه، اتصال کامل داده‌های تولیدشده توسط شبیه‌ساز به رابط کاربری و به‌روزرسانی لحظه‌ای مقادیر روی نمایشگر خواهد بود.